

Quest 1 Count pairs "ag"

Given a `char[]`, calculate the number of pairs (i, j) s.t. $i < j$ and $s[i] = 'a'$ & $s[j] = 'g'$.

All characters are lower case english alphabets. ('a' $\rightarrow$ 'z')

eg. 1.

0	1	2	3	4	5	6	7
b	a	a	g	d	c	a	g

pairs:- $\langle 1, 3 \rangle, \langle 1, 7 \rangle, \langle 2, 3 \rangle, \langle 2, 7 \rangle, \langle 6, 7 \rangle$

total pairs = 5

Quiz 1

0	1	2	3	4	5	6	7
b	c	a	g	g	a	a	g

total pairs:- 5

Quiz 2

0	1	2	3	4	5	6
a	c	g	d	g	a	g

total pairs = 4

Idea 1 Check all pairs-

```
int c=0;
for(i=0; i<N; i++){
    for(j=i+1; j<N; j++){
        if(s[i]=='a' & s[j]=='g'){
            c++;
        }
    }
}
return c;
```

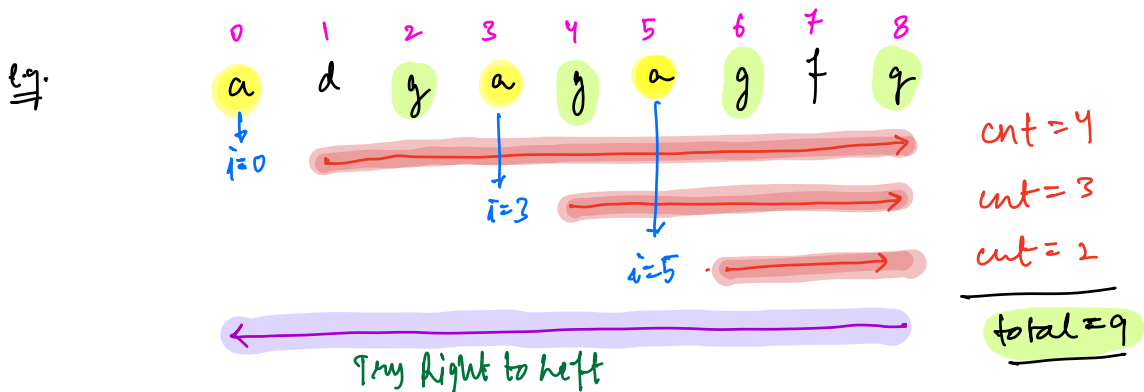
→ Don't need to worry about $i < j$.

T.C = $O(N^2)$
S.C = $O(1)$

Small optimisation.

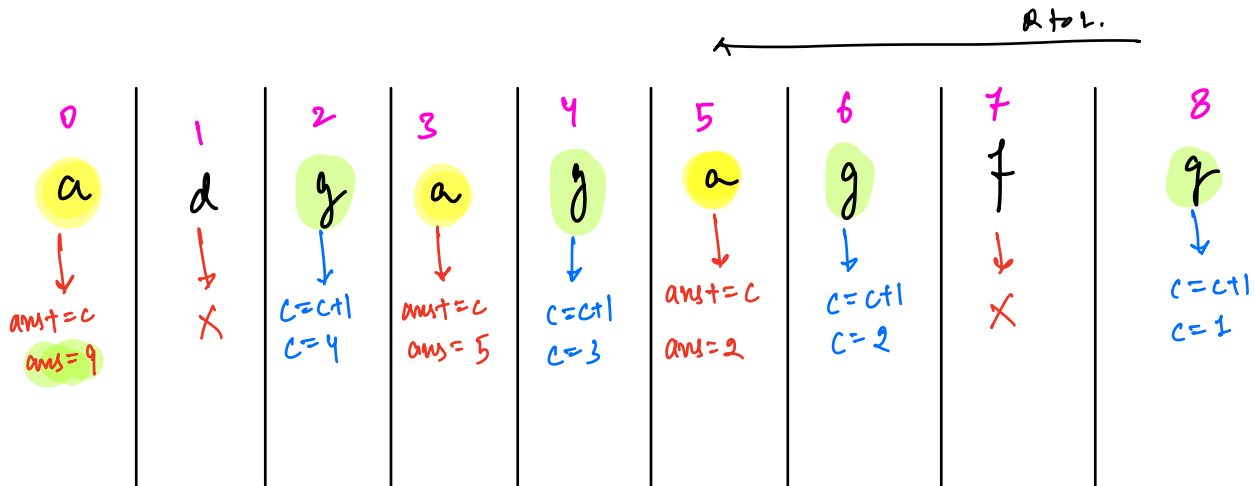
```
int c=0;
for(i=0; i<N; i++){
    if(s[i] != 'a')
        continue;
    for(j=i+1; j<N; j++){
        if(s[j] == 'g'){
            c++;
        }
    }
}
return c;
```

T.C = $O(N^2)$
S.C = $O(1)$



int ans = 0

int c = 0 → count of g's on right side of a



// Pseudo Code.

ans = 0; c = 0;

```
for (i = N-1; i >= 0; i--) {  
    if (arr[i] == 'g') { c++; }  
    else if (arr[i] == 'a') { ans += c; }  
}
```

return ans;

T.C = $O(N)$

S.C = $O(1)$

Interviewer says, move from left to right
and T.C = $O(N)$; S.C = $O(1)$

→ ans = 0

→ carry count of a from left to right, c.

→ when you find 'g' → ans += c;

// TODO.

Try out R to L and L to R both

Ques² Leaders in an Array

Given an array, you need to find leaders in an array.

An element is a leader if it is strictly greater than all elements on its right side.

↓
cannot be equal.

Note:- $arr[N-1]$ is always considered a leader.

e.g. 1:-

0	1	2	3	4	5	6	7
15	-1	7	2	5	4	2	3

→ ans = 5

Quiz arr[6]:-

0	1	2	3	4	5
10	7	9	3	2	4

→ ans = 3

Quiz arr[7]:-

0	1	2	3	4	5	6
8	-2	4	7	6	5	1

→ ans = 5

e.g. 2:- arr[3]:-

0	1	2
10	8	8

→ ans = 2

// Right to left → carry forward max value from right to left.

```
int c = 1;
int max = arr[N-1];
for (i = N-2; i >= 0; i--) {
    // arr[i] is leader or not
    if (arr[i] > max) {
        c++;
        max = arr[i];
    }
}
return c;
```

T.C = $O(N)$

S.C = $O(1)$

All edge cases handled.

Subarray Basis → continuous part of an array.

- 1) A single element can be a subarray
- 2) Entire array can also be a subarray
- 3) Empty array can not be a subarray.

eg. arr[9] =

0	1	2	3	4	5	6	7	8
-3	4	6	2	8	7	14	9	21

- 1) indices → [2, 3, 4, 5] → [2-5] → continuous part of an array.
- 2) indices → [3, 4, 6, 7, 8] → 5th index is missing
- 3) indices → [1, 2, 3] → subarray. → [1-3]
- 4) indices → [5] → [5-5] → subarray.

→ subarray → [3-7] → {3, 4, 5, 6, 7} → length = $7-3+1 = 5$

Generalisation:- subarray: [s, e] → length = $e-s+1$

// Using predefined functions.

- min(a, b) → returns min of a and b.
 - max(a, b) → returns max of a and b.
 - sort() → sorts data.
- } T.C = $O(1)$
T.C = $O(N \log N)$

Note:- Before you use any predefined fn. You should know it's T.C.

// Given N array elements, find min-val.

ans = arr[0]

for (i=0; i < N; i++) {

if (ans > arr[i]) {
ans = arr[i];
}

}

} → ans = min(ans, arr[i])

T.C = $O(N)$
S.C = $O(1)$

Break:- 10:40

Q3 Closest Min Max

Given an array. Find the length of smallest subarray which contains both min and max of array.

e.g.

0	1	2	3	4	5	6	7	8	9
1	2	3	1	3	4	6	4	6	3

min = 1

max = 6

indices:- $[3, 8] \rightarrow \text{len} = 8 - 3 + 1 \Rightarrow 6$
 $[3, 6] \rightarrow \text{len} = 6 - 3 + 1 \Rightarrow 4$ } $\rightarrow \text{ans} = 4$

Quiz

0	1	2	3	4	5	6	7	8	9	10
2	2	6	4	5	1	5	2	6	4	1

min = 1

max = 6

ind:- $[1, 7] \rightarrow \text{len} = 7$

ind:- $[2, 5] \rightarrow \text{len} = 4$

ind:- $[8, 10] \rightarrow \text{len} = 3 \rightarrow \text{this is ans.}$

Quiz

0	1	2	3	4	5
8	8	8	8	8	8

min = 8

max = 8

ind:- $[1, 1] \rightarrow \text{len} = 1$

Observations:-

- 1) There is only 1 min and 1 max in answer subarray.
- 2) Min and max are always present at the corners.

Min Max Min

--- Min --- Max ---

Case:- [Min --- Max]

[Max --- Min]

min=1, max=6

e.g.

ind:-	0	1	2	3	4	5	6	7	8	9	10	11	12
val:-	2	2	6	4	5	1	5	2	6	4	3	4	1
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
	X	X		X	X		X	X		X	X	X	X
			[2,5]			[5,8]			[8,12]				
			length = 5-2+1 = 4			length = 8-5+1 = 4			len = 5				

} → ans=4

// Pseudo code.

ans = N, 0, 1, arr[0], INT-MAX

iterate and get min-val of array

iterate and get max-val of array

```
if (min-val == max-val) {  
    return 1;  
}
```

```
for (i=0; i < N; i++) {
```

```
    if (arr[i] == minVal) { // find nearest max on right side
```

```
        for (j=i+1; j < N; j++) {
```

```
            if (arr[j] == min-val) { break; }
```

```
            if (arr[j] == max-val) {
```

```
                ans = min(ans, j-i+1)
```

```
                break;
```

```
            }
```

```
        }
```

```
    }  
    else if (arr[i] == max-val) { // find nearest min-val on the right side
```

```
        for (j=i+1; j < N; j++) {
```

```
            if (arr[j] == max-val) { break; }
```

```
            if (arr[j] == min-val) {
```

```
                ans = min(ans, j-i+1)
```

```
                break;
```

```
            }
```

```
        }
```

```
    }
```

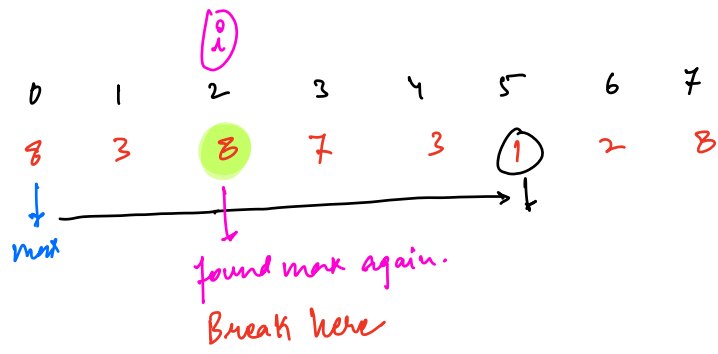
```
}
```

```
return ans;
```

T.C = $O(N^2)$ $\rightarrow O(N)$

S.C = $O(1)$

min=1
max=8



// Carry forward from R to L

→ min val index

→ max val index

→ ans = N

e.g.

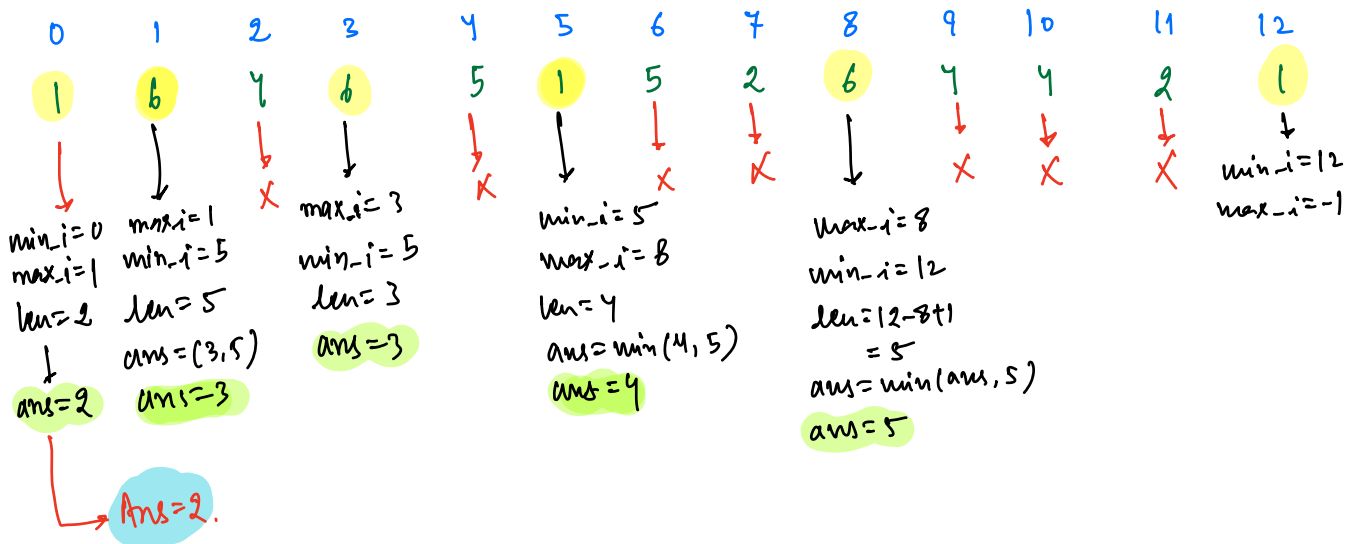
max_val = 6
min_val = 1

ans = N

max_i = -1

min_i = -1

← R to L



// Pseudo Code:-

1) iterate and get min-val, max-val
if (min-val == max-val) return 1;
ans = N, min-i = -1, max-i = -1;

```
for (i = N-1; i >= 0; i--) {  
    if (arr[i] == minVal) {  
        min-i = i;  
        if (max-i != -1) {  
            len = max-i - min-i + 1;  
            ans = min(ans, len);  
        }  
    }  
    else if (arr[i] == maxVal) {  
        max-i = i;  
        if (min-i != -1) {  
            len = min-i - max-i + 1;  
            ans = min(len, ans);  
        }  
    }  
}
```

$$T.C = O(N)$$

$$S.C = O(1)$$

[max-i min-i]

arr[] = { 1, 2, 3, 5, 10, 15 }

$$arr[0] + arr[1] = arr[1]$$

$$arr[1] + arr[2] = arr[2]$$

$$\underline{arr[i]} = arr[i-1] + \underline{arr[i]}$$